Mississippi Valley State University

CS 341 — Discrete Structures

Practice Set 3

Logical Equivalence and the Laws of Logic

Covers Chartrand & Zhang, Section 1.2, pp. 18–24

Status of this set

Ungraded practice. Not submitted; solutions released with it so you can check yourself. Attempt every problem before reading any solution.
Truth tables here have blank cells — fill them in on the page. Everything else goes on your own paper.

Part I: How to Practice This Material

The new skill is rewriting, not evaluating

Until now, every problem asked you to *compute* a truth value. This section asks something different: transform a statement into a different-looking statement that says the same thing. That is closer to algebra than to arithmetic, and it takes different practice.

Which tool for which job

Use a truth table when you are asked whether two statements are equivalent and you do not yet know. The table settles it either way.
Use the laws when you are asked to rewrite or simplify. A chain of named laws is shorter than a table, and it shows *why* the two forms agree rather than merely that they do.
Name the law at every step. "$\sim(P \wedge Q) \equiv (\sim P) \vee (\sim Q)$ by De Morgan" is a complete step. The same line with no name is an assertion, and it earns assertion-level credit.

Proving and disproving cost different amounts

To show $R \equiv S$, every row must agree — all 2^n of them.
To show $R \not\equiv S$, produce **one** row where they differ. You do not need the rest of the table, though building it is usually the fastest way to find that row.
Several problems below are disproofs. Do not do more work than the job requires, but do state the differing row explicitly — "they're not equivalent" without a witness is not an answer.

Four mistakes to expect

1. **Negating the pieces and leaving the connective alone.** The most expensive error in this section, and the one De Morgan exists to prevent.
2. **Dropping parentheses across different connectives.** $P \wedge Q \wedge R$ is fine. $P \wedge (Q \vee R)$ is not the same as $(P \wedge Q) \vee R$.
3. **Applying a law you have not checked.** A pattern that works for $\wedge$ and $\vee$ may fail for $\oplus$. Two problems below turn on exactly that.
4. **Skipping the law names in a simplification.** If a grader cannot tell which step you took, the step did not happen.

Part II: Practice Problems

Section A — Prove or disprove

1. Determine whether $\sim(P \vee Q)$ and $(\sim P) \vee (\sim Q)$ are logically equivalent. If they are not, name every row where they differ.

P	Q	$\sim(P \vee Q)$	$\sim P$	$(\sim P) \vee (\sim Q)$
T	T			
T	F			
F	T			
F	F			

2. Is $\oplus$ associative? That is, determine whether $P \oplus (Q \oplus R) \equiv (P \oplus Q) \oplus R$.

P	Q	R	$Q \oplus R$	$P \oplus (Q \oplus R)$	$P \oplus Q$	$(P \oplus Q) \oplus R$
T	T	T				
T	T	F				
T	F	T				
T	F	F				
F	T	T				
F	T	F				
F	F	T				
F	F	F				

3. Determine whether $P \vee (Q \oplus R) \equiv (P \vee Q) \oplus (P \vee R)$. If not, name every row where they differ.

P	Q	R	$Q \oplus R$	$P \vee (Q \oplus R)$	$P \vee Q$	$(P \vee Q) \oplus (P \vee R)$
T	T	T				
T	T	F				
T	F	T				
T	F	F				
F	T	T				
F	T	F				
F	F	T				
F	F	F				

Section B — Simplify using the laws

For each, produce a chain of equivalences ending in a statement with *no outer negation.* Name the law used at every step.

4. $\sim\big((\sim P) \vee Q\big)$

5. $\sim\big((\sim P) \wedge (\sim Q)\big)$

6. $\sim\big(\sim(P \wedge Q)\big)$

Section C — Negation in words

7. Write the negation of each statement, phrased the way a person would say it.

(a) The cache is warm and the query is indexed.

(b) Either n is prime or n is composite.

(c) $0 < x < 1$

(d) The user is neither an administrator nor the owner of the file.

(e) The list is sorted and contains no duplicates.

Section D — Rewriting conditions in code

8. A reviewer flags this condition as needlessly hard to read:

```
if not (not logged_in or not has_permission):
    grant_access()
```

Rewrite the condition in its simplest equivalent form, naming the laws you use. Then say in one sentence what the condition means.

9. For each pair of conditions below, the two operands are logically equivalent under the commutative law. Say whether the two operands can safely be *swapped* in the code, and explain what distinguishes the two cases.

```
(a) if index < len(items) and items[index] > 0:

(b) if x > 0 and y > 0:
```

Section E — Writing

10. In five to eight complete sentences, explain how both of the following can be true at once:

- $P \wedge Q \equiv Q \wedge P$ — the commutative law, proved by truth table.
- Swapping the two sides of an `and` in a program can crash it.

Your explanation must *not* claim that the commutative law is wrong or that it has exceptions. Refer to Problem 9 specifically.

Section F — Challenge (optional)

11. In Problem 3 you found that $\vee$ does *not* distribute over $\oplus$. Determine whether $\wedge$ does: is $P \wedge (Q \oplus R) \equiv (P \wedge Q) \oplus (P \wedge R)$?

12. De Morgan's Laws are stated for two statements. Verify the three-statement version: $\sim(P \vee Q \vee R) \equiv (\sim P) \wedge (\sim Q) \wedge (\sim R)$. Then say in one sentence what the general pattern must be for n statements.